AI TWITTER CONTENT GENERATOR

Technical Project Report

Project	twitter_content_app.py
Framework	Streamlit + Agno
AI Model	Llama 4 Scout via Groq
Report Date	April 19, 2026
Status	Development / Ready for Deployment

Confidential — Internal Use Only

1. Project Overview

The AI Twitter Content Generator is a Streamlit-based web application designed to automate the research and creation of high-quality Twitter/X content. By orchestrating multiple specialized AI agents powered by the Groq API, the application reduces the manual effort required to produce research-backed, hashtag-rich tweets — from hours to seconds.

At a Glance	
Purpose	Automated, research-backed Twitter/X content generation
Interface	Streamlit web application (browser-based UI)
AI Backend	Groq API — Meta Llama 4 Scout 17B model
Agent Framework	Agno multi-agent orchestration library
Output	Individual tweets or threaded tweet sequences
Character Support	Free (1,000 chars) & Paid (4,000 chars) accounts

2. System Architecture

The application follows a multi-agent pipeline pattern. Each agent is a specialised worker with defined tools and instructions. Agents run sequentially, passing research outputs downstream to enrich the final tweet generation step.

2.1 High-Level Data Flow

#	Stage	Agent	Tools Used
1	Web Search	Web Search Agent	DuckDuckGoTools
2	Deep Research	Deep Research Agent	DuckDuckGoTools (+ ReasoningTools*)
3	Competitor Analysis	Competitor Analyst	DuckDuckGoTools + FirecrawlTools
4	Tweet Writing	Tweet Writer	LLM only (no external tools)

** ReasoningTools is currently commented out in the codebase (pending integration).*

2.2 Core Technology Stack

Component	Technology	Role
UI / Frontend	Streamlit	Web app interface — sidebar, inputs, status widgets
Agent Orchestration	Agno	Defines and runs multi-agent pipelines
LLM Provider	Groq API	Ultra-fast inference for all agents
AI Model	Llama 4 Scout 17B	Reasoning, research & content generation
Web Search	DuckDuckGoTools	Fetches live web results for research
Web Scraping	FirecrawlTools	Scrapes competitor URLs (optional)
Data Validation	Pydantic v2	Structured output schemas (Tweet, TweetOutput)

3. Agent Descriptions

3.1 Web Search Agent

The first agent in the pipeline performs broad web reconnaissance on the user's topic using DuckDuckGo search. It identifies trending discussions, recent news, and key talking points to seed the research pipeline.

- Searches for current information and trending content
- Outputs a markdown summary of findings passed to downstream agents

3.2 Deep Research Agent

The Deep Research Agent performs multi-source analysis on the topic, cross-referencing information for accuracy and identifying unique angles. It is designed to think step-by-step and synthesise findings into structured insights.

- Cross-references multiple sources for factual accuracy
- Extracts key statistics, facts, and source citations
- Identifies controversy, debate points, and trending angles
- ReasoningTools integration is scaffolded (currently commented out)

3.3 Competitor Analyst

The Competitor Analyst is conditionally activated when the user provides competitor Twitter accounts or website URLs. If a Firecrawl API key is provided, it can scrape competitor websites for deeper content insights beyond social data.

- Analyses competitor content strategy and posting themes
- Identifies successful hashtag patterns
- Uses FirecrawlTools for deep website scraping when available
- Outputs actionable competitive intelligence for the tweet writer

3.4 Tweet Writer

The final agent in the chain synthesises all upstream research into polished, publish-ready tweets. It respects account character limits, generates 3–5 hashtags per tweet, and can produce either standalone tweets or connected threads.

- Enforces a minimum of 70% of the character limit per tweet
- Generates 3–5 relevant hashtags per tweet automatically
- Supports both individual tweet and thread formats
- Accepts a user-editable custom system prompt for tone/style control

4. Key Features

Feature Summary	
Research Pipeline	4-stage sequential agent pipeline with live status updates
Deep Research	Toggle-able in-depth analysis with multi-source synthesis
Competitor Analysis	Optional analysis with website scraping via Firecrawl
Tweet Formats	Individual tweets or connected thread sequences
Account Types	Supports Free (1,000 chars) and Paid (4,000 chars) limits
Smart Hashtags	Auto-generated, contextually relevant hashtags per tweet
Custom Prompts	Editable system prompt for tone and style customization
Download Output	One-click download of all generated tweets as a .txt file
API Key Management	User-provided keys — no hardcoded credentials in production

5. Pydantic Data Models

The application defines structured Pydantic models for type-safe output handling. These models are ready to be used with Agno's structured output capabilities and can enforce schema compliance on LLM responses.

5.1 Tweet

Field	Type	Description
content	str	The tweet text body
character_count	int	Computed character length of the tweet
hashtags	List[str]	List of associated hashtags (default: empty list)

5.2 TweetOutput

Field	Type	Description
tweets	List[Tweet]	All generated tweet objects
is_thread	bool	True when output is a connected thread
topic_summary	str	Brief summary of the researched topic

6. Security & Configuration

API keys are currently managed via environment variables and Streamlit input fields. The following observations and recommendations apply:

6.1 Current Implementation

- Groq and Firecrawl API keys are read from environment variables (os.getenv)
- Fallback values in code contain partial key strings — these must be removed before production deployment
- API keys are also accepted via the Streamlit sidebar at runtime — shown as password fields

6.2 Recommendations

- Remove all hardcoded or partial API key strings from the source code

- Use a .env file with python-dotenv or a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault)
- For cloud deployment (e.g., Streamlit Community Cloud), use the built-in Secrets management UI
- Add input validation to confirm API key format before making agent calls

7. Known Issues & Improvement Areas

Issue	Priority	Recommendation
Hardcoded partial API keys in code	Critical	Replace with environment-only secrets management
ReasoningTools commented out	Medium	Re-enable and test for improved research quality
Pydantic models not used for LLM output parsing	Medium	Wire TweetOutput as structured output schema in Agno
CompetitorInsight model unused	Low	Connect to competitor agent's response_model
Firecrawl limit=3 undocumented	Low	Document: limits number of pages crawled per request
No error handling for invalid API keys	Medium	Add try/except with user-friendly error messages
No caching of research results	Low	Add st.cache_data to avoid redundant API calls

8. Deployment Guide

8.1 Local Development

- **Install dependencies:** pip install streamlit agno pydantic
- **Search & scraping tools:** pip install duckduckgo-search firecrawl-py
- **Configure secrets:** Set GROQ_API_KEY and FIRECRAWL_API_KEY in a .env file or shell environment
- **Launch the app:** streamlit run twitter_content_app.py

8.2 Cloud Deployment (Streamlit Community Cloud)

- Push the repository to GitHub
- Connect the repo at share.streamlit.io
- Add GROQ_API_KEY and FIRECRAWL_API_KEY via the Streamlit Secrets panel
- Remove hardcoded fallback key values from the source before deploying

9. Summary

The AI Twitter Content Generator is a well-structured, modular application that effectively demonstrates multi-agent AI orchestration for real-world content automation. The pipeline architecture cleanly separates concerns — research, competitor intelligence, and writing — making it easy to extend or swap individual components.

Before production deployment, the two critical actions are: (1) removing all partial API keys from source code, and (2) wiring the existing Pydantic data models as structured output schemas in Agno for type-safe, parseable LLM responses. Both improvements are low-effort and high-impact.

With these changes in place, the application is well-positioned for deployment as a self-service social media content tool for individuals, marketing teams, or content agencies.